

PLU Module (CAEN V1495)

PLU Module (General Purpose VME Board CAEN V1495)

The Mod. V1495 is a VME 6U board, 1U wide, suitable for various digital Gate/Trigger/Translate/Buffer/Test applications, which can be directly customized by the User, and whose management is handled by two FPGAs:
FPGA "Bridge", which is used for the VME interface and for the connection between the VME interface and the 2nd FPGA
(FPGA "User") through a proprietary local bus.

FPGA "Bridge" manages also the programming via VME of the FPGA "User".
FPGA "User", which manages the front panel I/O channels.
FPGA "User" can be also free reprogrammed by the user with own custom logic function.
The FPGA "User" can be programmed "on the fly" directly via VME, without external hardware tools, without disconnecting the board from the set up, without resetting it or turning the crate off, allowing quick debug operations by the developer with his own firmware. A flash memory on the board can store two different programming files, which can be loaded to the FPGA "User" at any moment.

The inputs (A,B connectors) accept the LVDS or ECL signals level.
The output signals (C connector) have LVDS level and some LVDS/ECL translator is need to be used.

For our purposes we have developed a custom Programming Logic Unit (PLU) 16 inputs('A' connector) by 16 outputs ('C' connector)
on the base of V1495 module. Any number of inputs can be programmed in logic 'AND' or logic 'OR' to one of the output.

The logic of operation of the module type of LeCroy-2365 has been taken as base for this PLU project.

The project files are located under moller account on central machine in directory:

`~/moller/altera/project/V1495_USER_PLU16x16/`

The compilation of this project was implemented on Windows PC using Altera Quartus II version 6.0 program. All project files are located in directory: `C:\altera\V1495_USER_PLU16x16\`.

There is freely distributed version of this program Quartus II Web Edition (at this time version 7.2).

Quartus II Web Edition Project

The freely available Altera Quartus II (it can be downloaded from the Altera Web site) software must be used in order to generate a user firmware for the USER FPGA. It includes the source of VHDL reference design, which can be modified according to the description provided with the manual, in order to modify the card functionalities.

The tool provides a complete pinout of the FPGA; it is also enabled to generate the file type of programming (RBF format) used for the flash programming.

This software tool requires the Quartus II Web Edition rel. 5.1 (and newer) and can be freely downloaded at:

http://www.caen.it/nuclear/software_download.php

Quartus II manual is available at: www.altera.com/literature/hb/qts

The following figure shows the typical project flow for generating the firmware for an ALTERA FPGA, through the following steps:

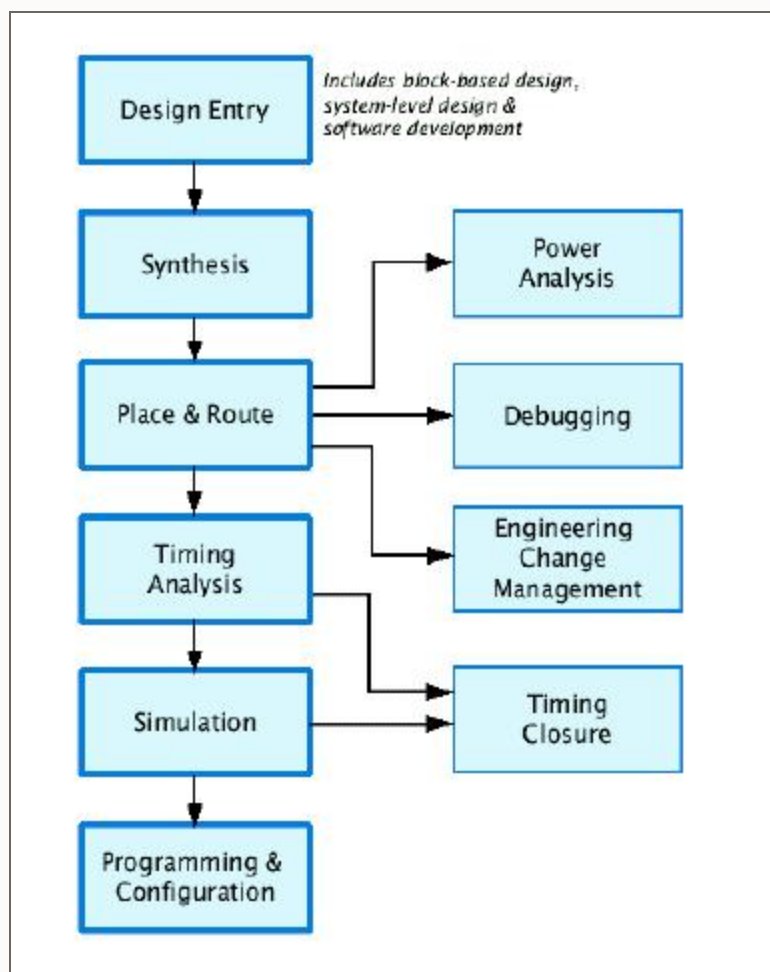
Design Entry is the functional description of the circuit; it could be either a description of the hardware (VHDL, Verilog, AHDL) or a scheme made with the tool provide by Quartus. The reference design provided is developed through VHDL; a VHDL knowledge is required in order to modify this design. A different description can be developed with a different language among those allowed by the Quartus tool.

Synthesis translates the description into a format compatible with the subsequent place&route step.

Place&route, starting from the netlist, performs the placing (place) and the subsequent interconnection (route) of the FPGA capabilities.

Simulation and timing analysis allow to verify the functionality of the project. The reference design includes a minimum set of constraints in order to allow the design to perform the foreseen function.

The last important step is the generation of the programming file. Quartus allows to generate different formats: the RBF format is the one used to program the FPGA USER via VME. The provided reference project produces automatically this format in the project directory under the filename v1495usr_demo.rbf.



Quartus II Project Flow.

The some files from demo project "coin_reference" from CAEN have been used for our custom PLU project.

There are 4 VHDL files under "V1495_USER_PLU16x16/SRC" directory (*.vhd) and a Verilog netlist (*.vqm):

The reference design of PLU project is included in the "usr_plu16x16.vhd" file. The file "usr_pkg.vhd" implements global constants (ID Codes, Register Addresses) for PLU project. The file "PLU16x16.qsf" describes assignments of internal signals to external FPGA pins and other project

options, in particular the list of the project files. Don't change pins assignment in this file. The other files provide support to the project and shall not be modified by the developer. HAL (Hardware Abstraction Layer) is implemented on the netlist Verilog "v1495usr_hal.vqm". The list of PLU project files located under directory V1495_USER_PLU16x16 are next:

```
src/v1495hal/v1495usr_hal.vqm
src/v1495usr_demo/tristate_if_rtl.vhd
src/v1495usr_demo/spare_if_rtl.vhd
src/v1495usr_demo/v1495usr_pkg.vhd
src/v1495usr_demo/PLU16x16.vhd
usr_plu16x16.vhd
usr_pkg.vhd
PLU16x16.qsf
PLU16x16.pin
PLU16x16.sof
PLU16x16.rbf
PLU16x16.pof
```

Flashing procedures

All source files for flashing of FPGA are located on 'adaqel2' computer in directory:
~/moller/dev/v1495

Flashing firmware (FPGA "Bridge") procedure:

1. Make login to VME CPU board using 'rlogin' or 'telnet' if portserver is used.
2. Change directory where file v1495.o is located by typing in VxWorks shell:
 - `cd "full_path_to_file"`
3. Download file v1495.o to VME CPU by typing command from VxWorks shell:
 - `ld < v1495.o`
4. Writing (flashing) to module firmware image by typing:
 - `v1495firmware(0x10100000,"V1495vme01.rbf",0,1)`where:
 - first parameter is module address on CPU local bus.
(this address can be get by system call:
`sysBusToLocalAdrs(VME_AM_EXT_USR_DATA, BASEADDRESS ,
&local_addr);`
`printf("%x\n",local_addr);`
 - where:
VME_AM_EXT_USR_DATA - address modifier (= 0x09 - A32, non-priv data access)
 - 'BASEADDRESS' is address that defined by rotary switches on module board (0x8100000)
 - local_addr - returned local address (0x10100000 for our VME CPU board type of VME23603)),
 - second parameter is file name with firmware image.This operation takes few minutes. Waiting until it is finished.

5. Reset(or power cycle) the module.

Flashing user image (FPGA "User") procedure (PLU 16X16 module):

(If previous steps are done, skip pp.1-3)

1. Make login to VME CPU board using 'rlogin' or 'telnet' if portserver is used.
2. Change directory where file v1495.o is located type in VxWorks shell:
 - `cd "full_path_to_file"`
3. Download file v1495.o to VME CPU by command from VxWorks shell:
 - `ld < v1495.o`
4. Change directory where file PLU16x16.rbf is located type in VxWorks shell:
 - `cd "full_path_to_file"`
5. Writing(flashing) to module user image by typing:
 - `v1495firmware(0x10100000,"PLU16x16.rbf",0,0)`
 where the first parameter is module address on CPU local bus.
 (this address can be get by system call:
`sysBusToLocalAdrs(VME_AM_EXT_USR_DATA, BASEADDRESS , &local_addr);`
`printf("%x\n",local_addr);`
 where:
 VME_AM_EXT_USR_DATA - address modifier (= 0x09 - A32, non-priv data access)
 'BASEADDRESS' is address that defined by rotary switches on module board (0x8100000)
 local_addr - returned local address (0x10100000 for our VME CPU board type of VME23603)),
 second parameter is file name with user image.
 This operation takes few minutes. Wait until it is finished.
6. Reset(or power cycle) the module.

Register Address Map of PLU16x16

Address	Default Value	Description
Base+0x001E	0x0000	Mode Operation Register (need to be set to 0x0000 before module operation bits: 0-5 should be set to 0; 6 - Test Mode Operation: if this bit is set to 1, then module gets inputs from data in Test Register instead of front panel connector, 7 - Red LED Control: if it is set to 1, then front panel RED LED is ON and otherwise
Base+0x0048	0x0001	Version of firmware (0.1)
Base+0x004A	0x0000	Output Complement Register (if bit 0 equals 1, then output #0 is inverted)
Base+0x004C	0x0000	Logic AND Register for output #0 (every bit represented as input number, if bit =1 then corresponded input connected to output #0 with AND logic)
Base+0x004E	0x0000	Logic AND Register for output #1
Base+0x0050	0x0000	Logic AND Register for output #2
Base+0x0052	0x0000	Logic AND Register for output #3
Base+0x0054	0x0000	Logic AND Register for output #4
Base+0x0056	0x0000	Logic AND Register for output #5
Base+0x0058	0x0000	Logic AND Register for output #6

Base+0x005A	0x0000	Logic AND Register for output #7
Base+0x005C	0x0000	Logic AND Register for output #8
Base+0x005E	0x0000	Logic AND Register for output #9
Base+0x0060	0x0000	Logic AND Register for output #10
Base+0x0062	0x0000	Logic AND Register for output #11
Base+0x0064	0x0000	Logic AND Register for output #12
Base+0x0066	0x0000	Logic AND Register for output #13
Base+0x0068	0x0000	Logic AND Register for output #14
Base+0x006A	0x0000	Logic AND Register for output #15
Base+0x006C	0x0000	Logic OR Register for output #0 (every bit represented as input number, if bit =1 then corresponded input connected to output #0 with OR logic)
Base+0x006E	0x0000	Logic OR Register for output #1
Base+0x0070	0x0000	Logic OR Register for output #2
Base+0x0072	0x0000	Logic OR Register for output #3
Base+0x0074	0x0000	Logic OR Register for output #4
Base+0x0076	0x0000	Logic OR Register for output #5
Base+0x0078	0x0000	Logic OR Register for output #6
Base+0x007A	0x0000	Logic OR Register for output #7
Base+0x007C	0x0000	Logic OR Register for output #8
Base+0x007E	0x0000	Logic OR Register for output #9
Base+0x0080	0x0000	Logic OR Register for output #10
Base+0x0082	0x0000	Logic OR Register for output #11
Base+0x0084	0x0000	Logic OR Register for output #12
Base+0x0086	0x0000	Logic OR Register for output #13
Base+0x0088	0x0000	Logic OR Register for output #14
Base+0x008A	0x0000	Logic OR Register for output #15
Base+0x008A	0x0000	Test Register (see description of Mode Operation Register)

Configuring of PLU for operation.

There are two options to configuring logic of operation of PLU. One of them is used routines from 'plu16x16.o' library in the coda '*.crl' file. The second one is remote configuration using 'plu_server' running on VME CPU and client program 'cli.pl' (perl script) (located in adaqel2:~/dev/vx/plu_server/).

To configuring the logic of the module operation next functions from 'plu16x16.o' library are used:

init_plu() - performs allocation of address and space for PLU register on VME bus.

Parameters: none;

Returns: 0 - on successful and -1 otherwise.

config_and(output_number, inputs_code) - configures AND logic for output 'output_number' and inputs in 'inputs_code'

Parameters: output_number - integer (0-15) output number to configure, inputs_code - integer (0x0000-0xFFFF) inputs numbers in format:

bit0-input#0, bit1-input#1, ..., bit15-input#15.

Returns: 0 on successful and -1 otherwise.

config_or(output_number, inputs_code) - configures OR logic for output 'output_number' and inputs in 'inputs_code'

Parameters: output_number - integer (0-15) output number to configure, inputs_code - integer (0x0000-0xFFFF) inputs numbers in format:

bit0-input#0, bit1-input#1, ..., bit15-input#15.

Returns: 0 on successful and -1 otherwise.

config_and_all(inputs_code) - configures AND logic for ALL outputs(0-15)

Parameters: output_number - inputs_code - integer (0x0000-0xFFFF) inputs numbers in format:

bit0-input#0, bit1-input#1, ..., bit15-input#15.

Returns: 0 on successful and -1 otherwise.

config_and_all(inputs_code) - configures OR logic for ALL outputs(0-15)

Parameters: output_number - inputs_code - integer (0x0000-0xFFFF) inputs numbers in format:

bit0-input#0, bit1-input#1, ..., bit15-input#15.

Returns: 0 on successful and -1 otherwise.

set_not(out) - inverted output for output number 'out'.

Parameters: out - output number(0-15);

Returns: 0 on successful and -1 otherwise.

config_test(data) - write 'data' to Test Register

Parameters: data - (0x0000-0xFFFF) data to write to Test Register;

Returns: 0 always;

set_mode(int mode) - sets mode of operation:

Parameters: mode - (0-1) mode of operation:

mode=1 - test mode(inputs data from Test Register);

mode=0 - normal mode(inputs data from front-end connector A);

Returns: 0 always;

get_inp() - returns data from input front-end connector A.

Parameters: none;

Returns: data from input front-end connector A.

get_out() - returns data from output front-end connector C.

Parameters: none;

Returns: data from output front-end connector C.

Examples:

Before using the configuration function 'init_plu()' should be executed.

1. To configuring inputs number 2,3,7 in AND logic to output number 0, inputs 0,2,5 in OR logic to output number 1:

```
config_and(0, 0x0046); /* AND inputs 2,3,7 to output#0 */
config_or(1, 0x0024); /* OR inputs 0,2,5 to output#1 */
```

Remote configuration.

For remote configuration of PLU the perl script 'cli.pl' or 'hvcli' program are used.

Format of configuration command for script 'cli.pl' is next:

```
cli.pl host port "message"
```

where:

host -is hostname (halladaq10.jlab.org) of the VME CPU where 'plu_server' is running

port -is port number (1357) to connect to the 'plu_server'.

"message" -is command string(in double or single quotes) to send to the 'plu_server'.

The message format of command string is next:

```
"destination command [parameter] [value1] [value2]"
```

where:

destination - module name from next list : PLU, TDC, DAC, VME, SERVER

command - command name: SET, CONFIG, GET, INIT, RESET, WRITE, READ

parameter - parameter name: AND, OR, NOT, TEST, OUT, STATUS

value1 - numeric value of the parameter (for PLU it is output channel number(0-15));

value2 - numeric value of the parameter (for PLU it is configuration word (0-65535) or in hex 0x0-0xFFFF);

The format of the respond message is next if command is successfully executed :

```
"OK <sent command message> [return value]"
```

On unsuccessfully execution the respond message is:

```
"ERROR <error description>"
```

The 'plu_server' supports next commands for PLU: CONFIG, GET, INIT
and next parameters: AND, OR, NOT, TEST

PLU commands description:

CONFIG - configuring logic AND or OR of inputs for selected output of PLU;

GET - get the configuration logic of AND or OR registers of PLU.

Examples:

Command message for configuring of PLU logic:

inputs #1 AND input#3 to output #0 :

```
cli.pl halladaq10 1357 "PLU config AND 0 5"
```

response message:

```
"OK PLU CONFIG AND 0 5"
```

or

```
cli.pl halladaq10 1357 "PLU config AND 0 0x0005"
```

response message:

```
"OK PLU CONFIG AND 0 0x0005"
```

inputs #2 OR input#7 to output #5 :

```
cli.pl halladaq10 1357 "plu CONFIG OR 5 65"
```

response message:

```
"OK PLU CONFIG OR 5 65"
```

or

```
cli.pl halladaq10 1357 "plu config OR 5 0x0041"
```

response message:

```
"OK PLU CONFIG OR 5 0x0041"
```

get configuration of AND registers for output #4 of PLU :

cli.pl halladaq10 1357 "plu GET AND 4"

response message:

"OK *PLU GET AND 4 0*"
